

**МИНОБРНАУКИ РОССИИ**  
**САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ**  
**ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ**  
**«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)**

**ОТЧЕТ**

**по лабораторной работе №2**

**по дисциплине «Построение и анализ алгоритмов»**

**ВАРИАНТ:** Вар. 4б. Добавляется 4-я операция со своей стоимостью: замена  
двух символов на один символ.

Студент гр. 4343

Зеров М.М.

Преподаватель

Жангиров Т.Р.

Санкт-Петербург

2026

## Задание

Расстоянием Левенштейна назовём минимальное количество операций вставки одного символа, удаления одного символа и замены одного символа на другой, необходимых для превращения одной строки в другую. Разработайте программу, осуществляющую поиск расстояния Левенштейна между двумя строками.

### Пример:

Для строк `pedestal` и `stien` расстояние Левенштейна равно 7:

- Сначала нужно совершить четыре операции удаления символа: `pedestal`  $\rightarrow$  `stal`.
- Затем необходимо заменить два последних символа: `stal`  $\rightarrow$  `stie`.
- Потом нужно добавить символ в конец строки: `stie`  $\rightarrow$  `stien`.

### Параметры входных данных:

Первая строка входных данных содержит строку из строчных латинских букв. ( $SS, 1 \leq |S| \leq 25501 \leq |S| \leq 2550$ ).

Вторая строка входных данных содержит строку из строчных латинских букв. ( $TT, 1 \leq |T| \leq 25501 \leq |T| \leq 2550$ ).

### Параметры выходных данных:

Одно число  $LL$ , равное расстоянию Левенштейна между строками  $SS$  и  $TT$ .

Тестовые данные

№ Теста

Входные данные

Выходные данные

1

7

`pedestal`

`stien`

## Описание алгоритма:

### Шаг 1: Инициализация

Создаётся двумерный массив  $dp$  размером  $(n+1)(m+1)$ , где  $n$  = длина строки  $A$ ,  $m$  = длина строки  $B$

Все ячейки инициализируются значением  $INF$  (бесконечность)

$dp[0][0]$  устанавливается в 0

### Шаг 2: Заполнение таблицы динамического программирования

Для всех  $i$  от 0 до  $n$ :

Для всех  $j$  от 0 до  $m$ :

Если  $dp[i][j] == INF$ , переход к следующей итерации

Иначе:

### Шаг 3: Обработка операций с одним символом

3.1: Совпадение: если  $i < n$ ,  $j < m$  и  $A[i] == B[j]$ , то  $dp[i+1][j+1] = \min(dp[i+1][j+1], dp[i][j])$

3.2: Замена: если  $i < n$ ,  $j < m$  и  $A[i] != B[j]$ , то  $dp[i+1][j+1] = \min(dp[i+1][j+1], dp[i][j] + \text{cost\_replace})$

3.3: Удаление: если  $i < n$ , то  $dp[i+1][j] = \min(dp[i+1][j], dp[i][j] + \text{cost\_delete})$

3.4: Вставка: если  $j < m$ , то  $dp[i][j+1] = \min(dp[i][j+1], dp[i][j] + \text{cost\_insert})$

### Шаг 4: Обработка операции замены двух символов на один (Replace2)

Если  $i + 2 \leq n$  и  $j + 1 \leq m$ :

$dp[i+2][j+1] = \min(dp[i+2][j+1], dp[i][j] + \text{cost\_replace2})$

### Шаг 5: Распространение оптимальных значений

Каждый переход обновляет ячейку, если найденное значение меньше текущего

### Шаг 6: Получение результата

Ответом является значение  $dp[n][m]$

Оно представляет минимальную стоимость преобразования всей строки  $A$  в строку  $B$

### Шаг 7: Завершение

Возвращается  $dp[n][m]$  - минимальная стоимость операций

**Оценка сложности алгоритма Левенштейна с операцией замены двух символов на один:**

Длина строки A —  $n$ ;

Длина строки B —  $m$ ;

Количество операций  $(n+1)(m+1)$

Может выполняться 5 переходов:

Совпадение

Замена

Удаление

Вставка

Замена двух одинаковых символов

Тогда общее количество итераций  $O(nm)$

Доп память: таблица  $dp$  размера  $(n+1)(m+1)$

Тесты:

user@WIN-675PLRMNU5F:~/pia\$ ./pr

```
=====
Запуск тестов для алгоритма Левенштейна
с операцией замены двух символов на один
=====
```

```
[=====] Running 11 tests from 1 test suite.
[-----] Global test environment set-up.
[-----] 11 tests from LevenshteinReplace2Test
[ RUN    ] LevenshteinReplace2Test.IdenticalStrings
```

```
>>> Запуск теста: IdenticalStrings (совпадающие строки) ✓ OK <<<
[ OK ] LevenshteinReplace2Test.IdenticalStrings (0 ms)
[ RUN  ] LevenshteinReplace2Test.SingleReplacement
```

```
>>> Запуск теста: SingleReplacement (одиочная замена) ✓ OK <<<
[ OK ] LevenshteinReplace2Test.SingleReplacement (0 ms)
[ RUN  ] LevenshteinReplace2Test.ReplaceTwoCharsWithOne
```

```
>>> Запуск теста: ReplaceTwoCharsWithOne (замена 2 символов на 1) ✓ OK
<<<
[ OK ] LevenshteinReplace2Test.ReplaceTwoCharsWithOne (0 ms)
[ RUN  ] LevenshteinReplace2Test.DeleteAndInsert
```

```
>>> Запуск теста: DeleteAndInsert (удаление и вставка) ✓ OK <<<
[ OK ] LevenshteinReplace2Test.DeleteAndInsert (0 ms)
```

[ RUN ] LevenshteinReplace2Test.Replace2Cheaper

>>> Запуск теста: Replace2Cheaper (замена 2 на 1 выгоднее) ✓ OK <<<

[ OK ] LevenshteinReplace2Test.Replace2Cheaper (0 ms)

[ RUN ] LevenshteinReplace2Test.ComplexExample

>>> Запуск теста: ComplexExample (сложный пример) ✓ OK <<<

[ OK ] LevenshteinReplace2Test.ComplexExample (0 ms)

[ RUN ] LevenshteinReplace2Test.EmptyStrings

>>> Запуск теста: EmptyStrings (пустые строки) ✓ OK <<<

[ OK ] LevenshteinReplace2Test.EmptyStrings (0 ms)

[ RUN ] LevenshteinReplace2Test.ReplaceTwoAtBeginning

>>> Запуск теста: ReplaceTwoAtBeginning (замена 2 символов в начале) ✓  
OK <<<

[ OK ] LevenshteinReplace2Test.ReplaceTwoAtBeginning (0 ms)

[ RUN ] LevenshteinReplace2Test.ReplaceTwoAtEnd

>>> Запуск теста: ReplaceTwoAtEnd (замена 2 символов в конце) ✓ OK <<<

[ OK ] LevenshteinReplace2Test.ReplaceTwoAtEnd (0 ms)

[ RUN ] LevenshteinReplace2Test.ReplaceTwoMultipleTimes

>>> Запуск теста: ReplaceTwoMultipleTimes (многократная замена 2 на 1) ✓  
OK <<<

[ OK ] LevenshteinReplace2Test.ReplaceTwoMultipleTimes (0 ms)

[ RUN ] LevenshteinReplace2Test.Replace2VsDeleteInsert

>>> Запуск теста: Replace2VsDeleteInsert (сравнение Replace2 с Delete+Insert)

✓ OK <<<

[ OK ] LevenshteinReplace2Test.Replace2VsDeleteInsert (0 ms)

[-----] 11 tests from LevenshteinReplace2Test (0 ms total)

[-----] Global test environment tear-down

[=====] 11 tests from 1 test suite ran. (0 ms total)

[ PASSED ] 11 tests.

=====

✓ ВСЕ ТЕСТЫ ПРОЙДЕНЫ УСПЕШНО

=====

user@WIN-675PLRMNU5F:~/pia\$